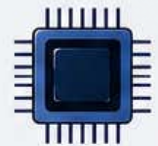
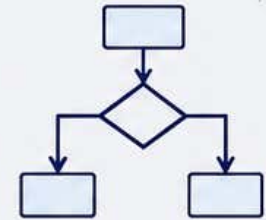


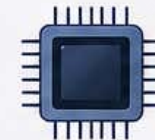
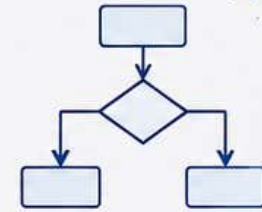
1010101010
0101010101
1010101010
0101010101



Notes



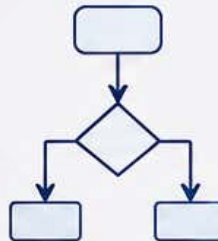
1010101010
0101010101
1010101010
0101010101



Unit-I



10110
01001
10101



BASIC TERMINOLOGY OF WEB AND WEB DESIGN PRINCIPLES

A. BASIC TERMINOLOGY OF WEB



WWW (World Wide Web) : A system of interlinked web pages and resources available on the Internet.



Internet : A global network of computers connected to each other for communication and sharing information.



Website : A collection of related web pages identified by a common domain name and published on a web server.



Web Page : A single document on the web written in HTML and viewed in a browser.



Web Browser : A software application used to access and view websites. (e.g., Chrome, Firefox, Edge)



Web Server : A computer that stores website files and delivers web pages to users' browsers.



Domain Name : The unique address of a website on the Internet. (e.g., www.example.com)



Hyperlink : A clickable link on a webpage that takes the user to another page or resource.



URL (Uniform Resource Locator) : The full address of a resource on the web. (e.g., https://www.example.com)

B. WEB DESIGN PRINCIPLES

Good web design makes a website useful, usable and attractive.



1. Purpose

Every website should have a clear purpose and goal.



2. User-Centered Design

Design for the users. Understand their needs and make the site easy to use.



3. Simplicity

Keep the design simple. Avoid clutter and unnecessary elements.



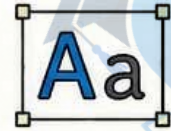
4. Consistency

Use consistent layout, colors, fonts and style across all pages.



5. Color Harmony

Use colors that are easy on the eyes and match the purpose of the site.



6. Readability

Use legible fonts, proper font size, good contrast and sufficient spacing.



7. Responsiveness

The website should work well on all devices and screen sizes.



8. Fast Loading

A good website should load quickly for a better user experience.



9. Accessibility

Design websites that can be used by people with different abilities.



Good web design = Attractive Look + Easy Navigation + Useful Content + Fast Performance + Positive User Experience



WHAT IS INTERNET ?

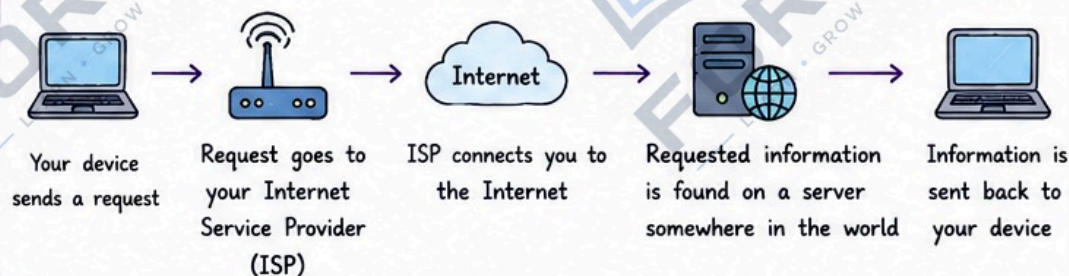
The Internet is a **global network** of interconnected computers and devices that communicate with each other using standard communication protocols. It allows people to share information, access services, and communicate from anywhere in the world.



Key Features of Internet

- **Global Connectivity** : Connects millions of devices worldwide.
- **Information Sharing** : Provides access to a vast amount of information.
- **Communication** : Enables email, chat, video calls, social media, etc.
- **Resource Sharing** : Sharing of hardware, software and data.
- **24x7 Access** : Available anytime, anywhere.

How Internet Works (In Simple Words)



Examples of Internet Services

- ★ World Wide Web (www)
- ★ E-mail
- ★ Online Banking
- ★ Social Networking
- ★ E-learning
- ★ Cloud Storage, etc.



BRIEF HISTORY OF INTERNET

- **1960s** The idea of a connected network came during the Cold War. The U.S. Department of Defense created **ARPANET** in 1969 to connect research computers.
- **1969** ARPANET was launched, connecting the first 4 computers in the USA on 29 October.
- **1970s** ARPANET grew and new protocols were developed. **E-mail** was introduced in 1971.
- **1983** **TCP/IP** protocol was adopted as the standard language of the Internet. This is considered the "birth of the Internet".
- **1984** The Domain Name System (**DNS**) was introduced, making it easier to remember addresses.
- **1990** Tim Berners-Lee invented the **World Wide Web (www)**. The first website went live.
- **1990s** Internet became public and commercial. Web browsers like **Mosaic** and **Netscape** were developed.
- **2000s** Broadband Internet, search engines (**Google**), social media and online services grew rapidly.
- **2010s** Smartphones, cloud computing, streaming, IoT and high-speed Internet became widespread.
- **2020s** Internet continues to evolve with **5G**, **AI**, big data and more smart, fast and secure services.

★ **In Short** : From ARPANET (1969) to today's global digital world, the Internet has changed the way we live, learn, work and connect.

WHAT IS WORLD WIDE WEB (WWW)?

The **World Wide Web (WWW)** is a system of interlinked hypertext documents and resources, accessible over the Internet. It uses **HTTP** protocol and web browsers to access information such as text, images, videos, etc.

Key Points

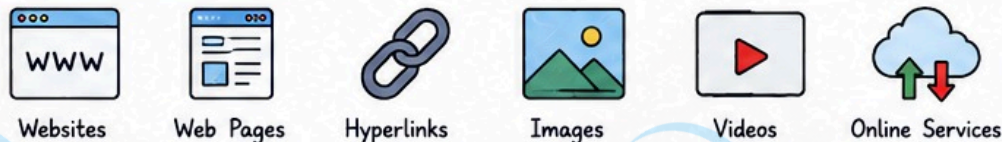
- ★ It is a part of the Internet.
- ★ Information on the Web is stored in web pages and websites.
- ★ Web pages are written in HTML and linked with hyperlinks.
- ★ We access the Web using a web browser (e.g., Chrome, Firefox, Edge).
- ★ WWW was invented by Tim Berners-Lee in 1989-1990 at CERN.



How WWW Works (In Simple Words)



Examples of WWW



💡 In short: The WWW is how we access and view information on the Internet using linked web pages through a browser.

CLIENT SERVER ARCHITECTURE

Client-Server Architecture is a model in which work is divided between two parts: the **client** (which requests services) and the **server** (which provides services). They communicate over a network (usually the Internet).

Main Components

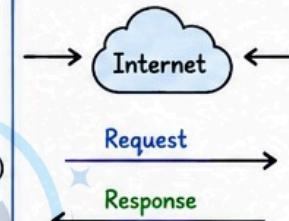
Client

The client is the user's device (or software) that requests information or services.

Examples:

- Web Browser (Chrome, Firefox)
- Mobile Apps
- Desktop Applications

Communication Over Network (Internet)



Server

The server is a powerful computer that stores data, processes requests and provides the requested information or services.

Examples:

- Web Server (Apache, Nginx)
- Database Server
- File Server, Mail Server

How Client-Server Architecture Works

1. Client



The client sends a request to the server for some resource (web page, data, etc.).

2. Server



The server receives the request, processes it and finds the required data or resource.

3. Response



The server sends the response back to the client. The client displays the result to the user.

Advantages

- ✓ Efficient use of resources
- ✓ Centralized data management
- ✓ Better security and data control
- ✓ Can handle many clients at the same time

Examples in Real Life

- When you open a website, your browser (client) requests the page and the web server sends it to you.
- Online banking, email, cloud storage, etc., all use client-server architecture.

★ In short: The client asks, the server processes and provides the result.

WEB SERVER Vs WEB BROWSER

Web server stores, processes and delivers web pages, while web browser requests, receives and displays them for the user.

WEB SERVER

A web server is a computer or software that **stores** websites and **delivers** web pages to users' browsers over the Internet.

Key Points

- Stores website files (HTML, CSS, JS, images, etc.).
- Processes client requests.
- Sends the requested web pages or data to the browser.
- Works 24/7 and handles many requests from different users.
- Uses protocols like HTTP/HTTPS.

Examples

- Apache HTTP Server
- Nginx
- Microsoft IIS
- LiteSpeed

Role

The web server is like a library. It stores books (web pages) and gives them to anyone who asks (the browser).



Vs

WEB BROWSER

A web browser is a software application that **requests** web pages from the server and **displays** them to the user.

Key Points

- Acts on behalf of the user.
- Sends requests to the web server for web pages or resources.
- Receives web pages (HTML, CSS, JS, images, etc.) from the server.
- Renders and displays the content in a readable form.
- Allows user interaction with the website.

Examples

- Google Chrome
- Mozilla Firefox
- Microsoft Edge
- Safari
- Opera

Role

The web browser is like a reader. It asks for a book (web page), gets it from the library (server) and shows it to you.



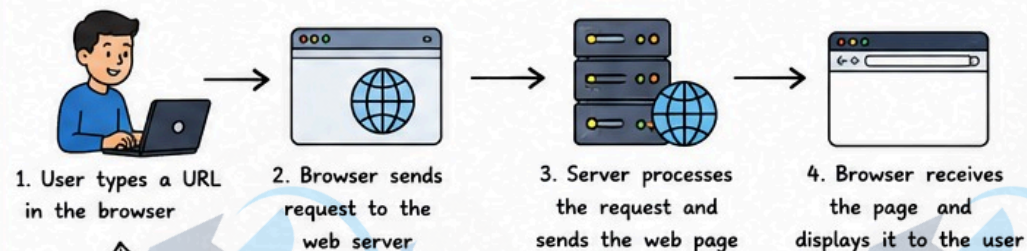
Web Browser

(Requests, Receives and Displays Web Pages)

KEY DIFFERENCES

Feature	Web Server	Web Browser
Definition	Stores and delivers web pages.	Requests and displays web pages.
Role	Provides resources.	Consumes resources.
Who Uses It?	Website owners, organizations.	End users.
Function	Processes requests and sends response.	Sends requests and renders response.
Examples	Apache, Nginx, IIS	Chrome, Firefox, Edge
Runs On	Powerful computers/servers.	User's device (PC, laptop, mobile).
Always Active?	Yes (24/7 to serve requests)	No (opened when user needs)

HOW THEY WORK TOGETHER

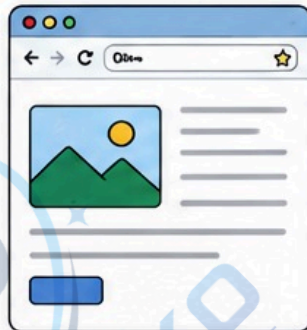


★ **In Short:** Web server is the **provider** of web pages, while web browser is the **viewer** of web pages.

WEBPAGES, WEBSITE, HOW WEBSITE WORKS?

1. WEBPAGES

A **webpage** is a single document on the World Wide Web. It is written in HTML and can contain text, images, links, videos, forms and other media.



Key Points

- One webpage = one file (e.g., index.html)
- Has a unique URL (address)
- Displayed in a web browser
- Can be static or dynamic

Examples of Webpages

- Home page
- About us page
- Contact us page
- Blog post page
- Product page



In Short

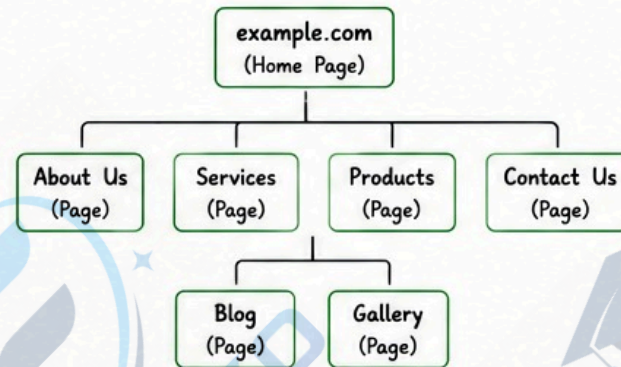
Webpage : Single page on the web.

Website : Collection of related webpages under one domain.

How it works : Browser communicates with the server to get and display webpages.

2. WEBSITE

A website is a collection of related webpages, images, videos and other resources, usually under one domain name, and published on a web server.



Key Points

- A website contains multiple webpages.
- All pages are linked together through hyperlinks.
- Identified by a domain name (e.g., www.example.com)
- Stored on a web server and accessible via the Internet.

Examples of Websites



3. HOW WEBSITE WORKS?

When you type a website address in the browser, the following steps happen:



1 You enter a URL

You type a website address (e.g., www.example.com) in the browser and press Enter.



2 DNS looks up the domain

The Domain Name System (DNS) converts the domain name into the IP address of the server.



3 Browser sends a request

The browser sends an HTTP/HTTPS request to the web server using that IP address.



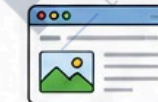
4 Server processes the request

The web server receives the request, processes it and finds the required webpage or resource.



5 Server sends a response

The server sends back the webpage (HTML, CSS, JS, images, etc.) to the browser.



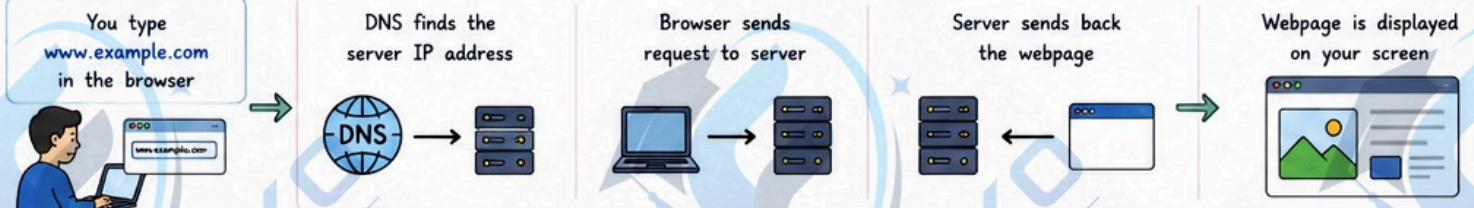
6 Browser displays the webpage

The browser reads the code, arranges the content and shows the webpage on your screen.

Simple Flow:

URL → DNS → Request → Server → Response → Webpage Display

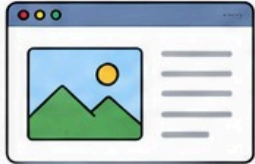
Example



TYPES OF WEBSITES

Websites can be different based on their purpose, content, and functionality.

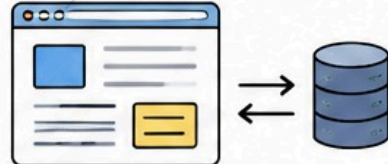
1. STATIC WEBSITES



- Pages are written in HTML and CSS.
- Same content for every user.
- No database or server-side processing.
- Load faster and easy to host.

Examples: Portfolio, Brochure websites

2. DYNAMIC WEBSITES



- Content changes based on user or time.
- Uses databases and server-side languages (e.g., PHP, Python, ASP.NET).
- More interactive and feature-rich.

Examples: News sites, Social media, E-commerce sites

3. E-COMMERCE WEBSITES



- Allow buying and selling of products online.
- Features: product catalog, cart, payment gateway, order tracking.
- Require secure transactions and databases.

Examples: Amazon, Flipkart, Shopify stores

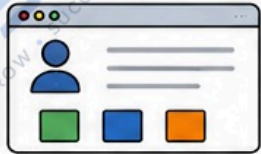
4. BLOG WEBSITES



- Regularly updated with articles or posts.
- Display content in reverse chronological order.
- Encourage user engagement through comments.

Examples: Medium, WordPress blogs

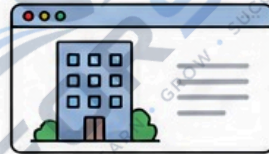
5. PORTFOLIO WEBSITES



- Showcases personal work, skills and achievements.
- Used by professionals like designers, developers, photographers, writers, etc.
- Helps in building personal brand.

Examples: Personal portfolio, Resume website

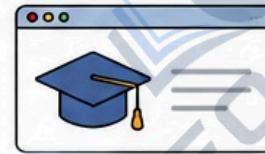
6. BUSINESS WEBSITES



- Provide information about a company, products or services.
- Includes pages like Home, About Us, Services, Contact Us.
- Helps in building online presence and trust.

Examples: Company websites, Service websites

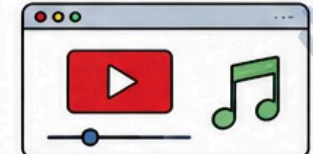
7. EDUCATIONAL WEBSITES



- Provide learning materials, tutorials, courses or resources.
- May include videos, quizzes, notes, forums.
- Used by schools, colleges, teachers, trainers, or e-learning platforms.

Examples: Khan Academy, Coursera

8. ENTERTAINMENT WEBSITES



- Provide entertainment content like videos, music, games, movies, etc.
- Focus on user engagement and experience.
- Include streaming, downloads, and more.

Examples: YouTube, Netflix, Spotify

★ In Short:

Websites are built for different purposes. Choosing the right type depends on the goals, content and users of the site.

Other Types of Websites

- Forum Websites (discussion forums)
- Government Websites (public services)
- Non-profit Websites (NGOs, charities)
- Job Portal Websites (career opportunities)
- Social Networking Websites (Facebook, LinkedIn)
- Personal Websites (personal information, hobbies)

Note:

Many websites can be a combination of types. For example, an E-commerce site can also be a business website and a dynamic website.

DOMAIN NAME, SERVER AND UNIFORM RESOURCE LOCATOR (URL)

These three concepts work together to make websites accessible on the Internet.

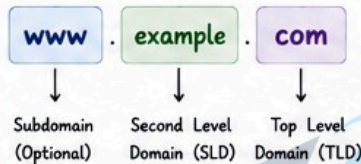
1. DOMAIN NAME

A **domain name** is the unique name given to a website that helps users find and remember a website easily instead of using IP addresses.

Key Points

- It is a human-readable address for a website.
- It is unique worldwide.
- Registered through domain registrars.
- Example: google.com

Structure of a Domain Name



Types of Top Level Domains (TLD)

Generic TLDs (gTLD)

.com, .org, .net, .info, .edu, .gov

Country Code TLDs (ccTLD)

.in (India), .uk (UK), .us (USA), .jp (Japan)

Example



★ **In Short:** Domain name is like the address name of your house on the Internet.

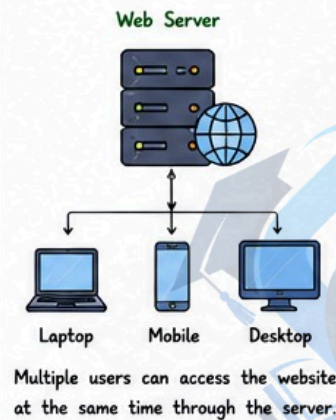
2. SERVER

A **server** is a computer or software that stores, processes and delivers website data or resources to users over the Internet.

Key Points

- Stores website files (HTML, images, videos, etc.).
- Processes requests from clients (browsers).
- Sends the requested data or web pages.
- Can be a physical computer or virtual machine.
- Always connected to the Internet.

How a Server Works



Types of Servers



★ **In Short:** Server is like the computer in which your website lives and from where it is delivered.

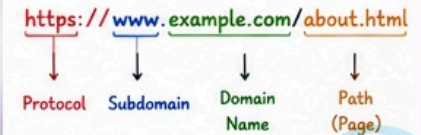
3. UNIFORM RESOURCE LOCATOR (URL)

A URL (Uniform Resource Locator) is the complete address of a web resource on the Internet that tells the browser where and what to fetch.

Key Points

- Gives the exact location of a web page or resource.
- Used by browsers to request and access resources.
- Every URL is unique.
- Example: <https://www.example.com/about.html>

Parts of a URL



Common Protocols

- **http** : HyperText Transfer Protocol (Not Secure)
- **https** : HyperText Transfer Protocol Secure (Secure)

Examples of URLs and Their Meanings

1. <https://www.google.com>
→ Home page of Google
2. <https://www.wikipedia.org/wiki/Internet>
→ Internet page on Wikipedia
3. <https://www.example.com/about.html>
→ About page of example website
4. <https://mail.google.com>
→ Gmail login page



★ **In Short:** URL is like the complete address of a house (protocol + domain + path).

How They Work Together



You type a URL in the browser



Domain Name

(e.g., www.example.com)
DNS converts domain name to IP address



Server

Server receives the request and processes it



Website / Web Page



Server sends the web page back to the browser



Browser Displays



The browser displays the web page to you

Remember:

Domain name helps you find, Server delivers, URL shows the exact location.

BASIC PRINCIPLES INVOLVED IN DEVELOPING A WEB SITE

These principles help create websites that are effective, attractive, easy to use and accessible to everyone.

1 PURPOSE & GOAL

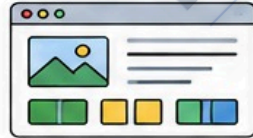


- Define the purpose of the website.
- Identify the target audience.
- Clear goals help in planning the content and features.

Example:

A school website aims to provide information to students, parents and staff.

2 SIMPLE & CLEAR DESIGN

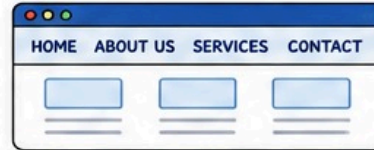


- Keep the design clean and simple.
- Avoid clutter and unnecessary elements.
- Use consistent colors, fonts and layouts.

Example:

Simple layout helps users focus on important information.

3 EASY NAVIGATION



- Navigation should be clear, consistent and easy to use.
- Users should find what they need in few clicks.
- Use menus, breadcrumbs and search option.

Example:

A well-structured menu helps users move around the site easily.

4 QUALITY CONTENT



- Provide accurate, useful and up-to-date information.
- Content should be easy to read and well organized.
- Use headings, lists and short paragraphs.

Example:

Good content builds trust and keeps users engaged.

5 RESPONSIVE DESIGN



- Website should look good and work well on all devices (desktop, tablet, mobile).
- Use responsive layouts and flexible images.

Example:

Users can access the site easily from any device.

6 FAST LOADING SPEED



- Pages should load quickly.
- Optimize images, use clean code and reliable hosting.
- Fast websites improve user experience and SEO.

Example:

Users may leave a site if it takes too long to load.

7 CONSISTENCY



- Maintain consistency in design, colors, fonts and layout.
- All pages should have a similar look and feel.
- Helps in building a professional and trustworthy site.

Example:

Same header, footer and style on all pages.

8 ACCESSIBILITY



- Make the website usable for all people including those with disabilities.
- Use readable fonts, alt text for images, proper contrast and keyboard navigation.
- Follow accessibility standards (WCAG).

Example:

Alt text for images helps visually impaired users.

9 SECURITY



- Protect user data and privacy.
- Use HTTPS, secure forms and regular updates.
- Prevent attacks like SQL injection, hacking, malware, etc.

Example:

HTTPS shows a secure connection (lock icon in browser).

10 TESTING & MAINTENANCE



- Test the website on different browsers, devices and screen sizes.
- Fix errors and broken links.
- Update content and features regularly.

Example:

Regular updates keep the website secure and useful.

★ **In Short:** A good website is planned with a clear goal, designed simply, easy to navigate, loaded with quality content, works on all devices, loads fast, is accessible to all, is secure and is tested & updated regularly.



Web Site Development Process

1. Planning



Define goals, audience and requirements

2. Design



Create layout, navigation and visual design

3. Development



Write code and build the website

4. Testing



Check for errors, compatibility, speed and usability

5. Deployment



Upload the website to web server

6. Maintenance



Update content, fix issues and improve regularly

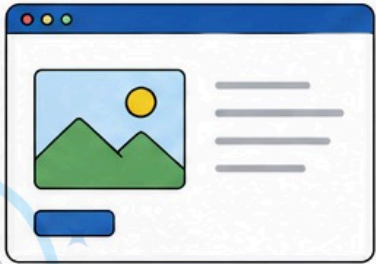


FIVE GOLDEN RULES OF WEB DESIGNING



Follow these rules to create websites that are effective, attractive and user-friendly.

1 KEEP IT SIMPLE



- Simplicity helps users understand your website easily.
- Avoid unnecessary elements, animations and clutter.
- Focus on important content and features.



Example:

A clean homepage with clear message and fewer distractions.

2 BE CONSISTENT



- Use consistent colors, fonts, layouts and styles on all pages.
- Consistency builds a professional look and improves usability.
- Keep the same header, menu and design pattern.



Example:

Same font, color scheme and buttons on every page.

3 MAKE IT EASY TO NAVIGATE



- Navigation should be clear, simple and user-friendly.
- Users must find what they need quickly and easily.
- Use menus, breadcrumbs and search option.



Example:

A well-structured menu helps users find information easily.

4 USE APPROPRIATE COLORS AND FONTS



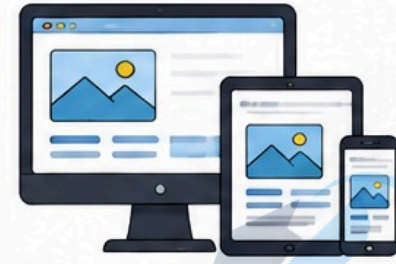
- Use colors that match your content and appeal to users.
- Ensure good contrast between background and text.
- Use readable fonts and limited font styles (2-3 fonts).



Example:

Dark text on light background is easy to read.

5 MAKE IT RESPONSIVE



- Your website should work well on all devices – desktop, tablet and mobile.
- Use responsive layouts and flexible images.
- It improves user experience and reaches a wider audience.



Example:

A website that looks great on mobile as well as desktop.



Remember: A good website is Simple, Consistent, Easy to Navigate, Visually Attractive and Responsive. These five golden rules help create websites that users love and remember.



WEB STANDARDS AND W3C RECOMMENDATIONS

Web standards ensure that websites are built using common rules so they can be used by everyone, on any device, with any browser.

1. WHAT ARE WEB STANDARDS?

Web standards are specifications and guidelines defined by recognized organizations to ensure quality, consistency, compatibility, accessibility and interoperability on the Web.

They help to:

- ✓ Make the web consistent and reliable
- ✓ Ensure websites work across different browsers and devices
- ✓ Improve accessibility for all users
- ✓ Promote innovation and growth of the Web



2. WHY WEB STANDARDS ARE IMPORTANT?



Cross-Browser Compatibility

Standards ensure websites work the same in all modern browsers.



Device Independence

Websites adapt to different screen sizes and devices.



Accessibility

Standards promote websites that are usable by people with disabilities.



Better SEO & Performance

Clean, standard-compliant code improves search engine ranking and website performance.



Future-Proof

Standards evolve with technology and keep your website up-to-date and maintainable.

3. WHAT IS W3C?



W3C (World Wide Web Consortium) is an international community that develops Web standards to lead the Web to its full potential.

W3C's mission:

"To lead the Web to its full potential by developing protocols and guidelines that ensure long-term growth for the Web."

4. HOW W3C WORKS?



Experts from around the world join W3C.



They discuss and create specifications.



Specifications are reviewed by the W3C community.



Final recommendations are published.

5. W3C RECOMMENDATION STATUS

Status	Meaning
Recommendation (REC)	The specification is mature and stable. It is the highest level of W3C endorsement.
Proposed Recommendation (PR)	Stable enough for implementation, but not final.
Candidate Recommendation (CR)	Ready for wide review and testing.
Working Draft (WD)	Early stage document. Work in progress.
Note / Informational (NOTE)	Provides information or guidance, not a standard.

6. IMPORTANT W3C STANDARDS AND RECOMMENDATIONS

STRUCTURE (HTML)



- HTML5
- XHTML
- HTML Living Standard

Defines the structure and content of web pages.

PRESENTATION (CSS)



- CSS3
- CSS
- Cascading Style Sheets Modules

Describes the look and layout of web pages.

BEHAVIOR (JAVASCRIPT)



- ECMAScript
- DOM (Document Object Model)
- Web APIs

Adds interactivity and dynamic behavior to web pages.

GRAPHICS & MEDIA



- SVG (Scalable Vector Graphics)
- Canvas 2D
- WebP, Media Formats

Standards for graphics, multimedia and visual content.

ACCESSIBILITY



- WAI (Web Accessibility Initiative)
- ARIA (Accessible Rich Internet Applications)

Makes the web accessible to people with disabilities.

OTHER STANDARDS



- URL, URI, IRI
- HTTP, HTTPS
- Web Security (Web Crypto)
- Web Storage
- Web Workers, etc.

Protocols and APIs that power the modern Web.



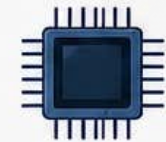
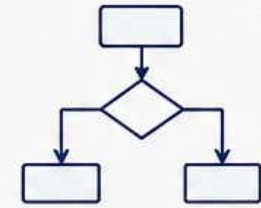
In Short:

- Web Standards = rules that every website should follow.
- W3C = the organization that creates and maintains those rules.
- Following W3C recommendations helps build websites that are compatible, accessible, fast, secure and future-ready.

Benefits of Following W3C Standards

- ✓ Works on all major browsers
- ✓ Reduced development time and cost
- ✓ Better user experience
- ✓ Supports accessibility and inclusivity

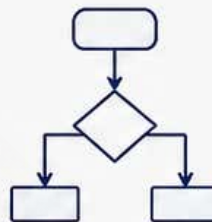
1010101010
0101010101
1010101010
0101010101



Unit - II



10110
01001
10101



HTML – The Standard Markup Language for Web Pages

1. WHAT IS HTML?

HTML stands for **HyperText Markup Language**.

It is the standard markup language used to create and structure content on the World Wide Web.

HTML tells the browser how to display text, images, links, tables, forms, videos and other content on a web page.

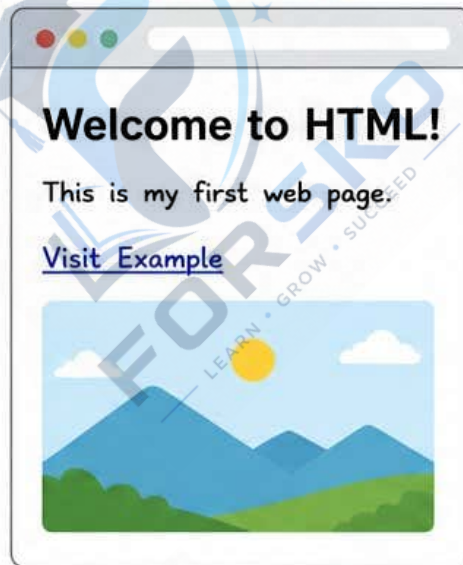
EXAMPLE:

HTML Code

```
<!DOCTYPE html>
<html>
  <head>
    <title>My First Page</title>
  </head>
  <body>
    <h1>Welcome to HTML!</h1>
    <p>This is my first
      web page.</p>
    <a href="https://www.example.com">
      Visit Example</a>
    <br>
    
  </body>
</html>
```



Output in Browser



Key Points

- ✓ HTML is not a programming language; it is a markup language.
- ✓ HTML works with CSS (for styling) and JavaScript (for behavior).
- ✓ HTML is supported by all web browsers.

2. FEATURES OF HTML

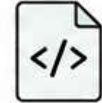
1.



Simple and Easy to Learn

HTML uses simple syntax and tags that are easy to understand and use.

2.



Platform Independent

HTML pages can be created on any platform (Windows, Linux, Mac) and viewed in any web browser.

3.



Browser Support

HTML is supported by all modern web browsers (Chrome, Firefox, Edge, Safari, etc.).

4.



Hypertext Capability

HTML allows linking one web page to another using hyperlinks.

5.



Supports Multimedia

HTML allows us to add images, audio, video and other multimedia content to web pages.

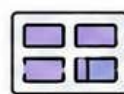
6.



Supports Forms

HTML provides form elements to collect input from users (text fields, radio buttons, checkboxes, etc.).

7.



Structure of Web Pages

HTML provides the structure or layout of a web page using elements like headings, paragraphs, lists, tables, etc.

8.



Extensible

HTML can be extended using CSS for styling and JavaScript for adding interactivity.

9.



Open Standard

HTML is an open standard maintained by the W3C (World Wide Web Consortium). Anyone can use it freely.

1. BASIC STRUCTURE OF AN HTML DOCUMENT

An HTML document has a specific structure. It begins with `<!DOCTYPE html>` and is enclosed within `<html>...</html>` tags.

EXAMPLE:

index.html

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport"
6     content="width=device-width,
7     initial-scale=1.0">
8   <title>My First Web Page</title>
9 </head>
10 <body>
11   <h1>Welcome to HTML!</h1>
12   <p>This is a paragraph.</p>
13   <a href="https://www.example.com">
14     Visit Example</a>
15 </body>
16 </html>
```

EXPLANATION OF THE STRUCTURE

`<!DOCTYPE html>`

Declares the document type and HTML version.

`<html>...</html>`

The root element that contains the entire HTML document.

`<head>...</head>`

Contains meta-information about the document such as title, character set, links to CSS, etc. (Not visible on the web page).

`<title>...</title>`

Sets the title of the web page. It appears on the browser tab.

`<body>...</body>`

Contains all the content that is visible on the web page, like headings, paragraphs, images, links, etc.

OUTPUT IN BROWSER



KEY POINTS

- ✓ HTML uses tags to define elements.
- ✓ Most elements have an opening tag and a closing tag.
- ✓ The content between the tags is displayed in the browser.

2. BASIC BUILDING BLOCKS OF HTML

1. HTML documents are made up of **elements**.

An element is the basic building block of an HTML page.

BUILDING BLOCK	DESCRIPTION	EXAMPLE	OUTPUT				
1. ELEMENT 	Everything in HTML is an element. It consists of an opening tag, content and a closing tag.	<code><p>Hello World!</p></code>	Hello World!				
2. TAG 	A tag is used to mark up the beginning and end of an element.	<code><h1>Heading</h1></code>	Heading				
3. ATTRIBUTES 	Attributes provide additional information about an element.	<code>Link</code>	Link				
4. HEADINGS 	Used to define headings. HTML has 6 levels (h1 to h6).	<code><h1>Main Heading</h1></code> <code><h3>Sub Heading</h3></code>	Main Heading Sub Heading				
5. PARAGRAPHS 	Used to define a block of text.	<code><p>This is a paragraph.</p></code>	This is a paragraph.				
6. LINKS 	Used to create links to other pages or resources.	<code>Go to Page</code>	Go to Page				
7. IMAGES 	Used to embed images in a web page.	<code></code>					
8. LISTS 	Used to display items in ordered or unordered lists.	<code></code> <code>Item 1</code> <code>Item 2</code> <code></code>	- Item 1 - Item 2				
9. TABLES 	Used to display data in rows and columns.	<code><table>...</table></code>	<table><tr><td>A</td><td>B</td></tr><tr><td>1</td><td>2</td></tr></table>	A	B	1	2
A	B						
1	2						
10. FORMS 	Used to collect input from users.	<code><form>...</form></code>	Name: <input type="submit" value="Submit"/>				

In Short: HTML provides the structure and content of a web page using elements like tags, attributes, headings, paragraphs, links, images, lists, tables and forms.

WORKING WITH LISTS, TABLES AND FRAMES

1

LISTS

Lists are used to display related items in a structured manner.

TYPES OF LISTS

- Unordered List – items marked with bullets.
- Ordered List – items marked with numbers.
- Description List – list of terms and their descriptions.

A. UNORDERED LIST ()

Syntax:

```
<ul>
  <li>Item 1</li>
  <li>Item 2</li>
  <li>Item 3</li>
</ul>
```

Output:

- Item 1
- Item 2
- Item 3

B. ORDERED LIST ()

Syntax:

```
<ol>
  <li>First</li>
  <li>Second</li>
  <li>Third</li>
</ol>
```

Output:

1. First
2. Second
3. Third

C. DESCRIPTION LIST (<dl>)

Syntax:

```
<dl>
  <dt>HTML</dt>
  <dd>Hyper Text Markup Language</dd>
  <dt>CSS</dt>
  <dd>Cascading Style Sheets</dd>
</dl>
```

Output:

HTML
Hyper Text Markup Language

CSS
Cascading Style Sheets



KEY POINTS

- ✓ Use for bullets, for numbers and <dl> for terms.
- ✓ Use to define list items.
- ✓ Lists help organize content and improve readability.

2

TABLES

Tables are used to display data in rows and columns.

BASIC TABLE STRUCTURE

```
<table border="1">
  <tr>
    <th>Header 1</th>
    <th>Header 2</th>
  </tr>
  <tr>
    <td>Data 1</td>
    <td>Data 2</td>
  </tr>
</table>
```

Output:

Header 1	Header 2
Data 1	Data 2

COMMON TABLE TAGS

- <table> Defines a table
- <tr> Defines a row in a table
- <th> Defines a header cell
- <td> Defines a data cell
- <caption> Adds a table title
- <thead> Groups header content
- <tbody> Groups body content
- <tfoot> Groups footer content

EXAMPLE: STUDENT MARKSHEET

Code:

```
<table border="1" cellpadding="5" cellspacing="0">
  <caption>Student Marks</caption>
  <tr style="background:#e8e8e8">
    <th>Roll No.</th><th>Name</th><th>Marks</th>
  </tr>
  <tr><td>1</td><td>Asha</td><td>85</td></tr>
  <tr><td>2</td><td>Ravi</td><td>78</td></tr>
  <tr><td>3</td><td>Neha</td><td>92</td></tr>
</table>
```

Output:

Roll No.	Name	Marks
1	Asha	85
2	Ravi	78
3	Neha	92



KEY POINTS

- ✓ Tables organize data in rows (<tr>) and columns (<th>, <td>).
- ✓ <th> is used for header cells (usually bold and centered).
- ✓ Use cellpadding and cellspacing for spacing in tables.

3

FRAMES

Frames divide the browser window into multiple sections, and each section can load a separate HTML document.

TYPES OF FRAMES

- Frameset – Defines the layout of frames.
- Frame – Specifies the HTML file to be displayed in a frame.
- No longer recommended in modern web design.

A. FRAMESET EXAMPLE (ROWS)

Code (frame_layout.html):

```
<html>
  <frameset rows="20%,60%,20%">
    <frame src="top.html" name="top">
    <frame src="main.html" name="main">
    <frame src="bottom.html" name="bottom">
  </frameset>
</html>
```

Output in Browser:

top.html
main.html
bottom.html

B. FRAMESET EXAMPLE (COLUMNS)

Code:

```
<html>
  <frameset cols="25%,75%">
    <frame src="menu.html" name="menu">
    <frame src="content.html" name="content">
  </frameset>
</html>
```

Output in Browser:

menu.html	content.html
-----------	--------------



KEY POINTS

- ✓ Frames were used to create multi-document web layouts.
- ✓ Each frame loads a separate HTML file.
- ✓ <frameset> is used instead of <body> when using frames.
- ✓ Frames are deprecated in HTML5. Use CSS layout or <iframe> instead.



Summary: Lists help in grouping items, Tables help in organizing data, and Frames help in dividing the browser window for multiple documents.



WORKING WITH TEXT, IMAGE & HYPERLINKS

1 WORKING WITH TEXT AND IMAGES

HTML allows us to display formatted text and images on a web page.

A. WORKING WITH TEXT

1. Headings

`<h1>Heading 1</h1>` `<h2>Heading 2</h2>` ... `<h6>Heading 6</h6>`

Code:

```
<h1>This is Heading 1</h1>
<h2>This is Heading 2</h2>
<h3>This is Heading 3</h3>
<h4>This is Heading 4</h4>
<h5>This is Heading 5</h5>
<h6>This is Heading 6</h6>
```

Output in Browser:

This is Heading 1
This is Heading 2
This is Heading 3
This is Heading 4
This is Heading 5
This is Heading 6

2. Paragraphs

`<p>This is a paragraph of text.
It is displayed in the browser.</p>` This is a paragraph of text. It is displayed in the browser.

3. Text Formatting

Tag	Purpose	Example & Output
<code>...</code>	Bold text	<code>Bold Text</code> → Bold Text
<code><i>...</i></code>	Italic text	<code><i>Italic Text</i></code> → <i>Italic Text</i>
<code><u>...</u></code>	Underlined text	<code><u>Underlined Text</u></code> → <u>Underlined Text</u>
<code>...</code>	Important text (bold)	<code>Important</code> → Important
<code>...</code>	Emphasized text (italic)	<code>Emphasis</code> → <i>Emphasis</i>
<code><sup>...</sup></code>	Superscript	<code>X<sup>2</sup></code> → X ²
<code><sub>...</sub></code>	Subscript	<code>H<sub>2</sub>O</code> → H ₂
<code>
</code>	Line Break	First Line → First Line (line break) Second Line
<code><hr></code>	Horizontal Rule	Adds a horizontal line.

B. WORKING WITH IMAGES

The `<img>` tag is used to embed an image in a web page.

Syntax:

```

```

Example:

```

```

Output in Browser:



KEY POINTS

- Use headings (`<h1>` to `<h6>`) for titles and subtitles.
- Use `<p>` for paragraphs and formatting tags to style text.
- Use `<img>` to add images. Always provide meaningful alt text for accessibility.

2 WORKING WITH HYPERLINKS

Hyperlinks are used to link one web page to another page, to a file, to an email address, or to a location within the same page.

A. THE `<a>` (ANCHOR) TAG

Syntax:

```
<a href="URL">Link Text</a>
```

Explanation:

- `<a>` is the anchor tag.
- `href` specifies the destination (URL).
- Link Text is the visible text that users click.

B. TYPES OF HYPERLINKS WITH EXAMPLES

Type		Code	Output in Browser
1. Link to Another Web Page		<code>About Us</code>	About Us
2. Link to a Website (External Link)		<code>Google</code>	Google
3. Link to an Image		<code>View Image</code>	View Image
4. Link to an Email Address		<code>Email Us</code>	Email Us
5. Link to a Section in the Same Page (Bookmark)		<code>Go to Contact Section</code>	Go to Contact Section
6. Link to a File (for Download)		<code>Download Notes</code>	Download Notes

C. EXAMPLE OF INTERNAL LINK (SAME PAGE)

Code:

```
<h2 id="top">Top of Page</h2>
<p>Welcome to our website!</p>
<p><a href="#bottom">Go to Bottom</a></p>
<hr>
<h2 id="bottom">Bottom of Page</h2>
<p>This is the bottom section.</p>
<p><a href="#top">Back to Top</a></p>
```

Output in Browser:

Top of Page

Welcome to our website!

[Go to Bottom](#)

Bottom of Page

This is the bottom section.

[Back to Top](#)



KEY POINTS

- Hyperlinks are created using the `<a>` tag.
- Use meaningful link text for better user experience and accessibility.
- `target="_blank"` opens the link in a new tab.
- Use internal links to navigate within the same page.

IMAGES AND MULTIMEDIA IN HTML

HTML allows us to add images, audio, video and other multimedia elements to make web pages more attractive and interactive.

1. IMAGES IN HTML

The `<img>` tag is used to embed an image in a web page.

Output in Browser

Syntax

```

```

Attributes

Attribute	Description
<code>src</code>	Specifies the path or URL of the image.
<code>alt</code>	Provides alternative text if the image cannot be displayed.
<code>width</code>	Sets the width of the image (in pixels).
<code>height</code>	Sets the height of the image (in pixels).



★ **Note:** Always use the `alt` attribute for better accessibility and SEO.

3. VIDEO IN HTML

The `<video>` tag is used to embed video content.

Output in Browser

Syntax

```
<video width="400" height="250" controls>
  <source src="movie.mp4" type="video/mp4">
  <source src="movie.webm" type="video/webm">
  <source src="movie.ogv" type="video/ogg">
  Your browser does not support the
  video tag.
</video>
```



Common Video Attributes

Attribute	Description
<code>controls</code>	Displays video controls (play, pause, seek, volume, etc.).
<code>autoplay</code>	Plays video automatically when the page loads.
<code>loop</code>	Plays the video in a loop.
<code>muted</code>	Mutes the video.
<code>poster</code>	Displays an image while the video is loading.
<code>width / height</code>	Sets the width and height of the video.

★ BEST PRACTICES

- ✓ Use relevant and optimized images to reduce page load time.
- ✓ Always provide `alt` text for images.

- ✓ Provide multiple source formats (MP3, OGG) for audio and (MP4, WebM, OGV) for video for better browser support.

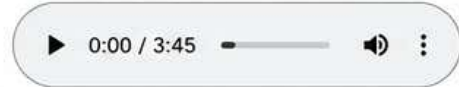
2. AUDIO IN HTML

The `<audio>` tag is used to embed sound content.

Syntax

```
<audio controls>
  <source src="song.mp3" type="audio/mpeg">
  <source src="song.ogg" type="audio/ogg">
  Your browser does not support the
  audio element.
</audio>
```

Output in Browser



Common Audio Attributes

Attribute	Description
<code>controls</code>	Displays audio controls (play, pause, volume, etc.).
<code>autoplay</code>	Plays audio automatically when the page loads.
<code>loop</code>	Plays the audio in a loop.
<code>muted</code>	Mutes the audio.

4. OTHER MULTIMEDIA ELEMENTS

A. EMBEDDING EXTERNAL CONTENT

The `<iframe>` tag is used to embed another web page within the current page.

Syntax

```
<iframe src="https://www.example.com"
width="600" height="400"
title="Example Website">
</iframe>
```

Output in Browser



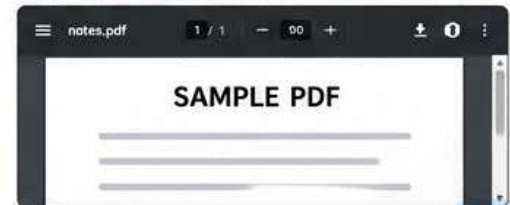
B. EMBEDDING PDF DOCUMENTS

PDF files can be embedded using `<embed>` or `<iframe>`.

Using `<embed>`:

```
<embed src="notes.pdf" type="application/pdf"
width="600" height="400">
```

Output in Browser



KEY TAKEAWAY

Images and multimedia make web pages engaging and interactive. Use them wisely to improve user experience.

- ✓ Use controls for better user experience.
- ✓ Do not use autoplay for media (it may disturb users).

WORKING WITH FORMS AND CONTROLS

HTML Forms are used to collect user input. The `<form>` element acts as a container for different form controls.

1. HTML FORM BASICS

A form is created using the `<form>` tag.

Syntax:

```
<form action="submit.php" method="post">
  <!-- form controls go here -->
</form>
```

Explanation:

- **action**: Specifies where to send the form data.
- **method**: Specifies how to send the data (GET or POST).

3. EXAMPLE: FORM CODE

Below is an example of a simple form using different controls.

```
<form action="register.php" method="post">
  <h3>Student Registration Form</h3>
  Name: <input type="text" name="name"><br><br>
  Password: <input type="password" name="pass"><br><br>
  Gender:
    <input type="radio" name="gender" value="M"> Male
    <input type="radio" name="gender" value="F"> Female<br><br>
  Hobbies:
    <input type="checkbox" name="hobby" value="Music"> Music
    <input type="checkbox" name="hobby" value="Sports"> Sports
    <input type="checkbox" name="hobby" value="Reading"> Reading<br><br>
  Course:
    <select name="course">
      <option value="">-- Select Course --</option>
      <option value="HTML">HTML</option>
      <option value="CSS">CSS</option>
      <option value="JS">JavaScript</option>
    </select><br><br>
  Upload Photo: <input type="file" name="photo"><br><br>
  Comments: <br>
    <textarea name="comment" rows="4" cols="30"></textarea><br><br>
    <input type="submit" value="Submit">
    <input type="reset" value="Reset">
</form>
```

2. FORM CONTROLS

Form controls are used to collect different types of user input.

Commonly used form controls:

Control	Input Type / Tag	Purpose	Example Code
	<code><input type="text"></code>	Single line text input	<code><input type="text" name="name"></code>
	<code><input type="password"></code>	Enters password	<code><input type="password" name="pass"></code>
	<code><textarea></code>	Multi-line text input	<code><textarea name="message"></textarea></code>
☐	<code><input type="radio"></code>	Select one option	<code><input type="radio" name="gender" value="M"></code>
☒	<code><input type="checkbox"></code>	Select one or more options	<code><input type="checkbox" name="hobby" value="Music"></code>
	<code><select><option></code>	Select one option from list	<code><select name="course"> <option>HTML</option> </select></code>
<input type="file"/>	<code><input type="file"></code>	Select a file to upload	<code><input type="file" name="file"></code>
<input type="submit"/>	<code><input type="submit"></code>	Submits the form data	<code><input type="submit" value="Submit"></code>
<input type="reset"/>	<code><input type="reset"></code>	Resets all form data	<code><input type="reset" value="Reset"></code>

4. IMPORTANT ATTRIBUTES

Attribute	Description
name	Specifies the name of the control (used to access data).
value	Specifies the initial value of a control.
placeholder	Provides a short hint that describes the expected value.
required	Makes the field mandatory.
readonly	Makes the field read-only.
disabled	Disables the control.
maxlength	Specifies the maximum number of characters allowed.
size	Specifies the visible width (in characters).

5. OUTPUT IN BROWSER

The form will appear in the browser as shown below.

Student Registration Form

Name:

Password:

Gender: ☐ Male ☐ Female

Hobbies: ☐ Music ☐ Sports ☐ Reading

Course:

-- Select Course --

Upload Photo: No file chosen

Comments:

POINTS TO REMEMBER

- `<form>` is the container for all form controls.
- Use appropriate input types for better user experience.
- Always use meaningful **name** attributes.
- Use **required** attribute to validate important fields.
- Submit button sends data to the server.



KEY POINTS

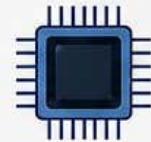
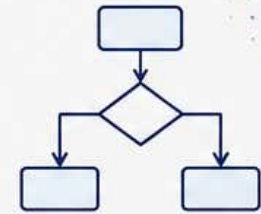
✓ Forms are used to collect user input.

✓ HTML provides a variety of controls to accept different types of data.

✓ Data is sent to the server using GET or POST method.

✓ Proper use of form controls improves usability and data accuracy.

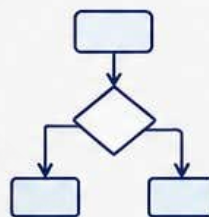
1010101010
0101010101
1010101010
0101010101



Unit - III



10110
01001
10101



CONCEPT OF CSS, NEED OF CSS

CSS stands for **Cascading Style Sheets**. It is used to style and layout web pages written in HTML.

1. CONCEPT OF CSS

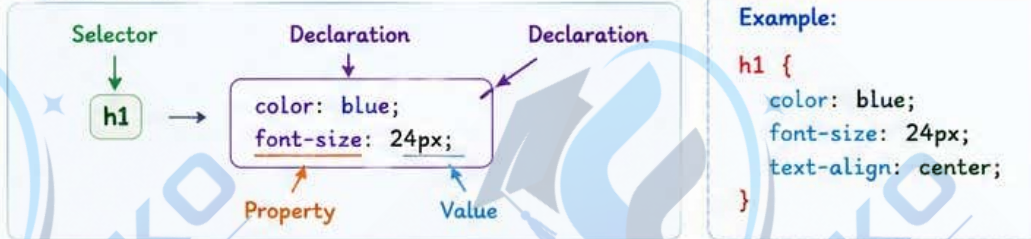
CSS is a style sheet language used to describe the presentation of a document written in HTML. It controls the look and feel of web pages.



What is CSS?

- CSS describes how HTML elements are to be displayed on screen, paper or other media.
- It saves a lot of work. It can control the layout of multiple web pages all at once.
- It enables you to apply styles consistently.

Syntax of CSS



How CSS works?



Ways to apply CSS

1. Inline CSS

Used inside an HTML element using style attribute.

```
<h1 style="color: blue;">  
  Hello World  
</h1>
```

2. Internal CSS

Used inside `<style>` tag in the `<head>` section.

```
<style>  
  h1 { color: blue; }  
</style>
```

3. External CSS

Used in a separate `.css` file and linked to HTML.

```
<link rel="stylesheet"  
      href="styles.css">
```



CSS follows the cascading order: **Inline** > **Internal** > **External**.

- The most specific style will be applied.

2. NEED OF CSS

CSS is used in web development to separate content from design. It helps in creating attractive, consistent and maintainable websites.



1. Improves Presentation

CSS allows you to apply colors, fonts, backgrounds, spacing, borders, animations and much more to make web pages visually appealing.



2. Consistency Across Pages

With CSS, the same style can be applied to multiple web pages, ensuring a consistent look and feel across the entire website.



3. Separates Content from Design

HTML is used for structure (content) and CSS for design (presentation). This separation makes the code clean and easy to manage.



4. Saves Time and Effort

You can change the style of the whole website by modifying a single CSS file instead of editing every HTML page.



5. Reduces Page Size & Improves Performance

CSS reduces the need for repeated HTML attributes and enables better browser caching, which improves loading speed.



6. Better Maintainability

CSS makes the website easier to maintain and update because design changes are done in one place.



7. Enhances Accessibility

CSS helps in creating user-friendly websites with proper readability, contrast, and responsive layouts for all devices.



8. Enables Responsive Design

CSS makes it easy to design responsive websites that work well on all screen sizes (desktop, tablet, mobile) using media queries.



In Short: CSS is the soul of a web page. It transforms a plain HTML page into a beautiful and responsive website.

Creating Style Sheet - Inline, External, Embedded CSS

CSS (Cascading Style Sheets) is used to add style and formatting to HTML documents. Styles can be applied in three ways: **Inline**, **External** and **Embedded**.

1. INLINE CSS

Inline CSS is written directly in the HTML tag using the style attribute. It is used for a single element.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Inline CSS</title>
  </head>
  <body>
    <h1 style="color:blue; text-align:center;">
      Welcome to My Website
    </h1>
  </body>
</html>
```

Welcome to My Website

Advantages:

- Quick and simple.
- Useful for one element.

Disadvantage:

- Not good for large websites.

Use: When you need to apply style to a single element.

2. EMBEDDED CSS

Embedded CSS is written inside the <style> tag in the <head> section of the HTML document. It is used to apply styles to the whole page.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Embedded CSS</title>
    <style>
      body { background-color: lightyellow; }
      h1 { color: green; }
      p { font-size: 18px; }
    </style>
  </head>
  <body>
    <h1>Welcome to My Website</h1>
    <p>This is a sample paragraph.</p>
  </body>
</html>
```

Welcome to My Website

This is a sample paragraph.

Advantages:

- Applies to the whole page.
- Keeps HTML and CSS together.

Disadvantage:

- Not reusable across multiple pages.

Use: When you want to style a single HTML page.

3. EXTERNAL CSS

External CSS is written in a separate .css file and linked to the HTML file using the <link> tag. It is the best method for large websites.

```
<!-- HTML file (index.html) -->
<!DOCTYPE html>
<html>
  <head>
    <title>External CSS</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <h1>Welcome to My Website</h1>
    <p>This is a sample paragraph.</p>
  </body>
</html>
```

```
/* style.css */
body { background-color: lightblue; }
h1 { color: navy; text-align: center; }
p { font-size: 16px; }
```

Welcome to My Website

This is a sample paragraph.

Advantages:

- Reusable across pages.
- Cleaner and organized code.
- Faster loading (cached).

Disadvantage:

- Requires an extra file.

Use: When you want to maintain a consistent style across multiple pages.

Quick Comparison

Feature	Inline CSS	Embedded CSS	External CSS
Where to write?	In HTML tag (style attribute)	In <style> tag (head section)	In separate .css file
Applies to	Single element	Whole page	Multiple pages
Code organization	Not good	Better	Best
Reusability	No	No	Yes
Recommended for	Small changes	Single page	Large websites

Key Points

- CSS stands for Cascading Style Sheets.
- It helps to make web pages attractive and user-friendly.
- Inline CSS uses the style attribute.
- Embedded CSS uses the <style> tag in the head section.
- External CSS uses a separate .css file linked with <link> tag.
- External CSS is the best practice for real-world websites.

CSS SELECTORS, TYPES OF SELECTORS

CSS Selectors are patterns used to select the HTML elements you want to style.
They tell the browser which element(s) the CSS rules will be applied to.

1. WHAT IS A CSS SELECTOR?

A CSS selector selects HTML element(s) and applies style to them.

Example:

```
p {  
  color: blue;  
}
```

HTML

```
<p>This is a paragraph.</p>  
<p>This is another paragraph.</p>
```



Result in Browser

This is a paragraph.

This is another paragraph.

2. TYPES OF CSS SELECTORS

Type of Selector	Syntax	Description	Example	Example Code	Result
1. Universal Selector (Selects all elements)	*	Selects all HTML elements on the page.	Selects all elements	<pre>* { margin: 0; padding: 0; }</pre>	Applies style to all elements.
2. Element (Type) Selector (Selects by element name)	p	Selects all elements with the specified tag name.	Selects all <p> elements	<pre>p { color: blue; }</pre>	Applies style to all <p> elements.
3. Class Selector (Selects by class)	.class	Selects all elements with the specified class attribute.	Selects all elements with class="note"	<pre>.note { background: yellow; }</pre>	<div class="note">Note content</div>
4. ID Selector (Selects by id)	#id	Selects the element with the specified id attribute.	Selects the element with id="header"	<pre>#header { color: green; }</pre>	Applies style to the element with id="header".
5. Attribute Selector (Selects by attribute)	[attribute] [attribute="value"]	Selects elements that have the specified attribute or attribute with value.	Selects all input elements type="text"	<pre>input[type="text"] { border: 1px solid #888; padding: 5px; }</pre>	
6. Grouping Selector (Selects multiple elements)	h1, p, .box	Selects multiple selectors and applies the same style.	Selects all <h1>, <p> and elements with class="box"	<pre>h1, p, .box { font-family: Arial; line-height: 1.6; }</pre>	Applies style to all selected elements.

3. MORE SELECTORS

Selector	Syntax	Description	Example	Example Code	Result
7. Descendant Selector (Selects descendant elements)	div p	Selects all <p> elements that are descendants of <div>.	Selects <p> inside <div>	<pre>div p { color: purple; }</pre>	<p>Text</p>
8. Child Selector (Selects direct children)	ul > li	Selects all elements that are direct children of .	Selects direct inside	<pre>ul > li { list-style: square; }</pre>	- Item 1 - Item 2 - Item 3
9. Pseudo-class Selector (Selects elements in a specific state)	a:hover	Selects <a> element when user hovers over it.	Selects a link on hover	<pre>a:hover { color: red; text-decoration: underline; }</pre>	Link

4. KEY POINTS

- Selectors are used to target HTML elements.
- Different selectors have different syntax and are used in different situations.
- You can combine selectors to be more specific.
- Using the right selector helps in writing clean, efficient and maintainable CSS.

EXAMPLE OF COMBINING SELECTORS

```
div#main .box h2 {  
  color: navy;  
  font-size: 20px;  
}
```

Selects <h2> elements inside elements with class "box" which are inside the element with id="main".



Remember: The more specific the selector, the higher its precedence.

Specificity Order (Low to High):

* < element < .class < #id < inline style

CSS PROPERTIES – POSITION, FLOAT, CLEAR, VISIBILITY

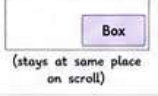
CSS properties control the layout and visibility of elements on a web page.

1. POSITION PROPERTY

The position property specifies how an element is positioned in a document.

Values:

- **static** (default) – Normal flow
- **relative** – Relative to its normal position
- **absolute** – Relative to nearest positioned ancestor
- **fixed** – Relative to browser window
- **sticky** – Acts like relative until a scroll position is reached, then becomes fixed

static (default)		<pre>.box { position: static; }</pre>
relative		<pre>.box { position: relative; top: 20px; left: 20px; }</pre>
absolute		<pre>.box { position: absolute; top: 10px; right: 10px; }</pre>
fixed		<pre>.box { position: fixed; bottom: 20px; right: 20px; }</pre>
sticky		<pre>.box { position: sticky; top: 0; }</pre>

Note: For absolute, fixed and sticky to work properly, the element may need a positioned ancestor.

2. FLOAT PROPERTY

The float property is used to take an element out of the normal flow and float it to the left or right of its container.

Values:

- **left** – Floats the element to the left
- **right** – Floats the element to the right
- **none** – Default (no floating)
- **inherit** – Inherits from parent

Example

```
img {
  float: left;
  margin: 10px;
}
```

Result:



This is a paragraph of text that wraps around the floated image. The image is floated to the left using CSS float property.

Use:

- To wrap text around images.
- To create multi-column layouts.
- To align elements side by side.



Floated elements are removed from the normal document flow.

3. CLEAR PROPERTY

The clear property specifies what elements can float beside the element and on which side.

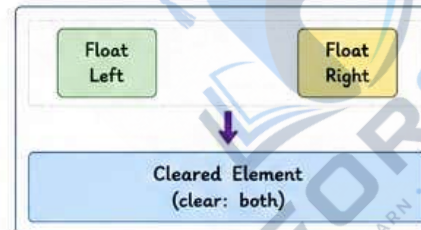
Values:

- **none** – Default (allow floating on both sides)
- **left** – No floating elements on the left side
- **right** – No floating elements on the right side
- **both** – No floating elements on either side
- **inherit** – Inherits from parent

Example

```
.content {
  clear: both;
}
```

How it works?



Result:

The element with `clear: both`; appears below both the left and right floated elements.

Use:

- To move an element below the floated elements.
- Commonly used after floated layouts.

4. VISIBILITY PROPERTY

The visibility property specifies whether an element is visible or hidden.

Values:

- **visible** – Element is visible (default)
- **hidden** – Element is hidden but still occupies space
- **collapse** – For table rows/columns, hides and removes space
- **inherit** – Inherits from parent

Examples

```
.box1 { visibility: visible; }
.box2 { visibility: hidden; }
.box3 { visibility: collapse; }
```

Result:

visible	I am Visible (takes space)												
hidden	(Hidden but still takes space)												
collapse (for table rows)	<table><tr><th>ID</th><th>Name</th><th>Mark</th></tr><tr><td>1</td><td>Ram</td><td>75</td></tr><tr><td colspan="3">— (collapsed row) —</td></tr><tr><td>2</td><td>Shyam</td><td>85</td></tr></table>	ID	Name	Mark	1	Ram	75	— (collapsed row) —			2	Shyam	85
ID	Name	Mark											
1	Ram	75											
— (collapsed row) —													
2	Shyam	85											

Use:

- To hide elements without removing them from layout.
- To show/hide elements using JavaScript or CSS.

QUICK SUMMARY

Property	Purpose	Common Values	Affects Layout Space?
position	Specifies how an element is positioned	static, relative, absolute, fixed, sticky	Yes (except static)
float	Floats an element to left or right	left, right, none, inherit	Yes (removes from flow)
clear	Prevents floating elements on sides	none, left, right, both, inherit	Yes (moves element below floats)
visibility	Shows or hides an element	visible, hidden, collapse, inherit	hidden = Yes, collapse = No

★ KEY POINTS

- position controls the placement of an element.
- float is used for creating layouts and text wrapping.
- clear is used to control elements below floated elements.
- visibility hides or shows elements but hidden still takes space.



Tip

To clear floats easily on modern layouts, you can use:

```
.clearfix::after {
  content: "";
  display: block;
  clear: both;
}
```

or use CSS overflow: auto;

CSS FLEXBOX LAYOUT, CSS GRID LAYOUT

CSS provides powerful layout systems to create flexible and responsive web designs.

1. CSS FLEXBOX LAYOUT

Flexbox is a one-dimensional layout model used for arranging items in rows or columns. It is ideal for small-scale layouts and components.

Key Concepts

- Flex Container** - The parent element. (display: flex)
- Flex Items** - The direct children of the container.

How it Works?



Flex Container Properties

Property	Purpose
display: flex;	Enables flexbox.
flex-direction	row row-reverse column column-reverse
justify-content	Aligns items along the main axis (horizontal in row).
align-items	Aligns items along the cross axis (vertical in row).
flex-wrap	wrap nowrap wrap-reverse (items wrap to next line)
gap	Sets space between items.

Flex Item Properties

Property	Purpose
order	Changes the visual order.
flex-grow	Defines how much item grows.
flex-shrink	Defines how much item shrinks.
flex-basis	Initial size of the item.
align-self	Overrides align-items for a particular item.

Example

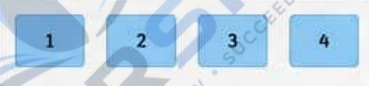
HTML

```
<div class="flex-container">
  <div class="item">1</div>
  <div class="item">2</div>
  <div class="item">3</div>
  <div class="item">4</div>
</div>
```

CSS

```
.flex-container {
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 15px;
  background: #f3f3f3;
  padding: 15px;
}
.item {
  background: #7ecbe3;
  padding: 12px 20px;
  border-radius: 6px;
  font-weight: bold;
}
```

Result



Common Use Cases

- Navigation bars
- Centering content
- Equal height columns
- Card layouts
- Form layouts

Quick Flexbox Example (Center Content)

HTML

```
<div class="center-box">
  <p>Hello Flexbox!</p>
</div>
```

CSS

```
.center-box {
  display: flex;
  justify-content: center;
  align-items: center;
  height: 120px;
  background: #eef6ff;
  border: 2px dashed #7aa7d9;
}
```

Result

Hello Flexbox!

Flexbox vs Grid

Feature	Flexbox	Grid
Type	One-dimensional (Row or Column)	Two-dimensional (Rows and Columns)
Best for	Small components, navigation, alignment	Complex layouts, page structure
Alignment	Along main axis or cross axis	Both rows and columns
Layout Control	Simple and flexible	More powerful and precise

2. CSS GRID LAYOUT

Grid is a two-dimensional layout model used for designing complex layouts. It works with rows and columns.

Key Concepts

- Grid Container** - The parent element. (display: grid)
- Grid Items** - The direct children of the container.

How it Works?



Grid Container Properties

Property	Purpose
display: grid;	Enables grid layout.
grid-template-columns	Defines number and size of columns.
grid-template-rows	Defines number and size of rows.
gap	Space between rows and columns.
justify-items	Aligns items horizontally in their cells.
align-items	Aligns items vertically in their cells.
place-items	Shorthand for align-items and justify-items.

Grid Item Properties

Property	Purpose
grid-column	Item position / span in columns.
grid-row	Item position / span in rows.
justify-self	Horizontal alignment inside the cell.
align-self	Vertical alignment inside the cell.

Example

HTML

```
<div class="grid-container">
  <div class="item">1</div>
  <div class="item">2</div>
  <div class="item">3</div>
  <div class="item">4</div>
  <div class="item">5</div>
  <div class="item">6</div>
</div>
```

CSS

```
.grid-container {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  grid-template-rows: 100px 100px;
  gap: 15px;
  background: #f3f3f3;
  padding: 15px;
}
.item {
  display: flex;
  justify-content: center;
  align-items: center;
  font-weight: bold;
  border-radius: 6px;
}
```

Result



Common Use Cases

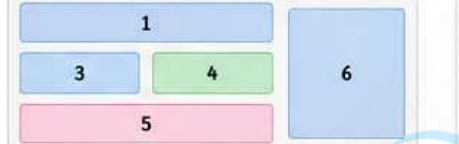
- Full page layouts
- Dashboard layouts
- Gallery layouts
- Complex web pages

Grid Item Spanning Example

CSS (Add to above)

```
.item:nth-child(1) {
  grid-column: span 2; /* item 1 spans 2 columns */
}
.item:nth-child(6) {
  grid-row: span 2; /* item 6 spans 2 rows */
}
```

Result



When to Use?

Use Flexbox when:

- You need to align items in a row or column.
- The layout is simple and linear.
- You want space distribution between items.



Use Grid when:

- You need both rows and columns.
- You want to design the overall page layout.
- You need items to span multiple rows or columns.



CSS STYLING PROPERTIES

(BACKGROUND, TEXT FORMAT, CONTROLLING FONTS)

CSS (Cascading Style Sheets) is used to describe the look and formatting of HTML elements. It controls **colors**, **backgrounds**, **text appearance** and **fonts** to make web pages attractive and readable.

1. BACKGROUND PROPERTIES

Background properties are used to set background color, images, position, size and behavior.

Property	Description	Example
background-color	Sets the background color of an element.	background-color: lightblue;
background-image	Sets one or more background images.	background-image: url('bg.jpg');
background-repeat	Sets how a background image repeats.	background-repeat: no-repeat;
background-position	Sets the starting position of a background image.	background-position: center top;
background-size	Sets the size of the background image.	background-size: cover;
background-attachment	Sets whether the background image scrolls with the page.	background-attachment: fixed;
background	Shorthand property to set all background properties in one declaration.	background: #f0f0f0 url('bg.jpg') no-repeat center / cover fixed;

EXAMPLE

CSS

```
.box {
  background-color: #e0f7fa;
  background-image: url('nature.jpg');
  background-repeat: no-repeat;
  background-position: center;
  background-size: cover;
  height: 200px;
  color: white;
  display: flex;
  justify-content: center;
  align-items: center;
  font-size: 22px;
  font-weight: bold;
}
```

RESULT



2. TEXT FORMAT PROPERTIES

Text formatting properties control the appearance and arrangement of text.

Property	Description	Example
color	Sets the color of text.	color: #333333;
text-align	Aligns text horizontally.	text-align: center;
text-decoration	Adds decoration to text (underline, overline, line-through or none).	text-decoration: underline;
text-transform	Controls capitalization of text.	text-transform: uppercase;
text-indent	Indents the first line of text.	text-indent: 30px;
letter-spacing	Sets space between characters.	letter-spacing: 2px;
word-spacing	Sets space between words.	word-spacing: 5px;
line-height	Sets the space between lines of text.	line-height: 1.6;
text-shadow	Adds shadow to text.	text-shadow: 2px 2px 4px gray;
white-space	Controls how white space inside an element is handled.	white-space: nowrap;

EXAMPLE

CSS

```
.text-box {
  color: #0d47a1;
  text-align: justify;
  text-decoration: none;
  text-transform: capitalize;
  letter-spacing: 1px;
  word-spacing: 3px;
  line-height: 1.7;
  text-indent: 40px;
  text-shadow: 1px 1px 2px #aaa;
}
```

RESULT

Css Styling Properties

Css is used to style web pages. It controls colors, text appearance, fonts and layout. Proper text formatting makes content easy to read and attractive.

3. CONTROLLING FONTS

Font properties control the typeface, size, weight and style of text.

Property	Description	Example
font-family	Sets the font type.	font-family: 'Arial', sans-serif;
font-size	Sets the size of text.	font-size: 18px;
font-weight	Sets how thick or thin text is.	font-weight: bold;
font-style	Sets font style (normal, italic or oblique).	font-style: italic;
font-variant	Sets small-caps text.	font-variant: small-caps;
line-height	Sets the space between lines.	line-height: 1.5;
font	Shorthand to set all font properties in one line.	font: italic bold 16px/1.5 'Roboto', sans-serif;

EXAMPLE

CSS

```
.font-box {
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
  font-size: 20px;
  font-weight: 600;
  font-style: italic;
  line-height: 1.6;
  color: #1a237e;
}
```

RESULT

Web Typography

Choosing the right font and size improves readability and gives your website a professional look.

COMMON WEB-SAFE FONTS

Category	Examples
Sans-serif	Arial, Verdana, Tahoma, Trebuchet MS, Segoe UI
Serif	Times New Roman, Georgia, Cambria, Palatino
Monospace	Courier New, Consolas, Monaco
Cursive	Brush Script MT, Lucida Handwriting
Fantasy	Impact, Comic Sans MS

Aa

QUICK NOTES

1. Use background shorthand to save time.
2. Use color names, HEX, RGB or HSL values.
3. Good background + text color combination improves readability.
4. Font choice and size affect user experience.

BEST PRACTICES

- Use consistent colors and fonts across the website.
- Ensure good contrast between background and text.
- Avoid too many font styles on one page.
- Use web-safe fonts or import fonts for better design.

WORKING WITH LISTS AND TABLES

HTML Lists are used to group related items. HTML Tables are used to display data in rows and columns.

1. WORKING WITH LISTS

HTML provides three types of lists:

1. Ordered List
2. Unordered List
3. Description (Definition) List

1.1 Ordered List ()

Used to display items in a numbered list.

HTML

```
<ol>
  <li>HTML</li>
  <li>CSS</li>
  <li>JavaScript</li>
</ol>
```

Result in Browser

1. HTML
2. CSS
3. JavaScript

Types of Ordered List

Use `type` attribute in `<ol>` to change numbering style.

- `type="1"` → 1, 2, 3 (default)
- `type="A"` → A, B, C
- `type="a"` → a, b, c
- `type="I"` → I, II, III
- `type="i"` → i, ii, iii

1.2 Unordered List ()

Used to display items in a bulleted list.

HTML

```
<ul>
  <li>Apple</li>
  <li>Mango</li>
  <li>Banana</li>
</ul>
```

Result in Browser

- Apple
- Mango
- Banana

Types of Unordered List

Use `type` attribute in `<ul>` to change bullet style.

- `type="disc"` • (default)
- `type="circle"` ○
- `type="square"` ■
- `type="none"` No bullets

1.3 Description (Definition) List (<dl>)

Used to display terms and their descriptions.

```
<dl>
  <dt>HTML</dt>
  <dd>HyperText Markup Language</dd>
  <dt>CSS</dt>
  <dd>Cascading Style Sheets</dd>
  <dt>JS</dt>
  <dd>JavaScript</dd>
</dl>
```

Result in Browser

HTML
HyperText Markup Language

CSS
Cascading Style Sheets

JS
JavaScript

2. WORKING WITH TABLES

HTML tables are used to organize data into rows and columns.

2.1 Table Structure

Tag	Description
<code><table></code>	Defines the table
<code><tr></code>	Defines a table row
<code><th></code>	Defines a header cell (bold & centered)
<code><td></code>	Defines a data cell
<code><caption></code>	Adds a title/caption for the table
<code><thead></code>	Groups the header content
<code><tbody></code>	Groups the body content
<code><tfoot></code>	Groups the footer content

Anatomy of a Table

<code><caption> Student Details </caption></code>		
<code><th> Roll No. </th></code>	<code><th> Name </th></code>	<code><th> Grade </th></code>
<code><td> 1 </td></code>	<code><td> Riya </td></code>	<code><td> A </td></code>
<code><td> 2 </td></code>	<code><td> Ali </td></code>	<code><td> B </td></code>
<code><td> 3 </td></code>	<code><td> Neha </td></code>	<code><td> A </td></code>
<code><td colspan="3"> Total Students: 3 </td></code>		

2.2 Example of Table

HTML Code

```
<table border="1" cellpadding="8" cellspacing="0" style="border-collapse: collapse;">
  <caption>Student Details</caption>
  <thead>
    <tr>
      <th>Roll No. </th>
      <th>Name</th>
      <th>Grade</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>1</td>
      <td>Riya</td>
      <td>A</td>
    </tr>
    <tr>
      <td>2</td>
      <td>Ali</td>
      <td>B</td>
    </tr>
    <tr>
      <td>3</td>
      <td>Neha</td>
      <td>A</td>
    </tr>
  </tbody>
  <tfoot>
    <tr>
      <td colspan="3" style="text-align: center; font-weight: bold;">
        Total Students: 3
      </td>
    </tr>
  </tfoot>
</table>
```

Result in Browser

Student Details		
Roll No.	Name	Grade
1	Riya	A
2	Ali	B
3	Neha	A
Total Students: 3		

Quick Notes

- ✓ Use lists to group related data in a meaningful way.
- ✓ Ordered list is for steps or sequence.
- ✓ Unordered list is for groups where order is not important.
- ✓ Description list is useful for definitions or glossary.

Tips

- ✓ Use `<th>` for header cells (bold & centered).
- ✓ Use `<caption>` to give a title to the table.
- ✓ Use `border`, `collpadding` and `collspacing` attributes for better appearance.
- ✓ Use `colspan` and `rowspan` to merge cells.

`colspan` (merge columns)

A	B
<code>colspan="2"</code>	

`rowspan` (merge rows)

A	B
<code>rowspan="2"</code>	C

CSS BOX MODEL

Every HTML element is treated as a **rectangular box**.
The Box Model consists of: **Content** → **Padding** → **Border** → **Margin**

1. BORDER PROPERTIES

Border properties are used to set the border of an element.

Property	Description	Example
border-width	Sets the width of the border.	border-width: 2px;
border-style	Sets the style of the border.	border-style: solid;
border-color	Sets the color of the border.	border-color: blue;
border	Shorthand property to set width, style and color.	border: 2px solid blue;
border-top	Sets all top border properties.	border-top: 2px dotted red;
border-right	Sets all right border properties.	border-right: 3px solid green;
border-bottom	Sets all bottom border properties.	border-bottom: 2px dashed orange;
border-left	Sets all left border properties.	border-left: 3px double purple;
border-radius	Rounds the corners of an element.	border-radius: 8px;

Example

```
.box {  
  border: 3px solid #3498db;  
  border-radius: 10px;  
  padding: 15px;  
}
```

Result in Browser

This is a box with
border.

2. PADDING PROPERTIES

Padding is the space between the content and the border.

Property	Description	Example
padding	Shorthand property for all padding properties.	padding: 20px;
padding-top	Adds padding to the top.	padding-top: 20px;
padding-right	Adds padding to the right.	padding-right: 30px;
padding-bottom	Adds padding to the bottom.	padding-bottom: 20px;
padding-left	Adds padding to the left.	padding-left: 30px;

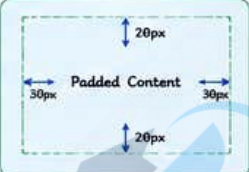
Values

- 1 value → all sides
- 2 values → top & bottom | left & right
- 3 values → top | left & right | bottom
- 4 values → top | right | bottom | left

Example

```
.box {  
  padding: 20px 30px 20px 30px;  
  background-color: #e8f6ff;  
  border: 2px solid #1abc9c;  
}
```

Result in Browser



3. MARGIN PROPERTIES

Margin is the space outside the border. It creates a bounding element.

Property	Description	Example
margin	Shorthand property for all margin properties.	margin: 25px;
margin-top	Adds margin to the top.	margin-top: 25px;
margin-right	Adds margin to the right.	margin-right: 25px;
margin-bottom	Adds margin to the bottom.	margin-bottom: 25px;
margin-left	Adds margin to the left.	margin-left: 25px;
margin: auto	Centers a block element horizontally.	margin: auto;

Example

```
.box {  
  width: 300px;  
  margin: 30px auto;  
  padding: 15px;  
  border: 2px solid #e67e22;  
  background-color: #fff8e1;  
}
```

Result in Browser



4. DISPLAY PROPERTIES

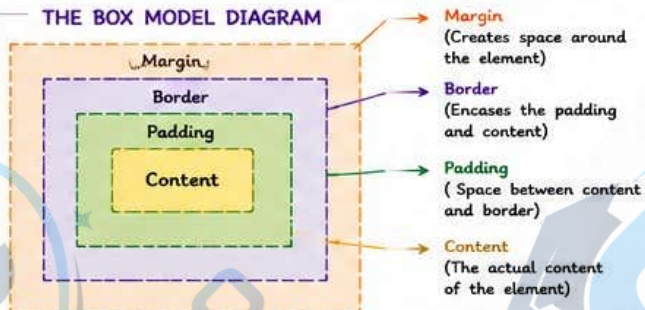
The display property specifies how an element is displayed on the page.

Value	Description	Example
block	Takes full width. Starts on a new line.	div, h1, p { display: block; }
inline	Takes only required width. Does not start on new line.	span, a { display: inline; }
inline-block	Takes only required width but allows setting width/height.	img { display: inline-block; }
none	Hides the element. It is not displayed.	.box { display: none; }
flex	Enables flex container for flex items.	.container { display: flex; }
grid	Enables grid container for grid items.	.container { display: grid; }

Display Examples

block	Element 1 Element 2
inline	Element 1 Element 2 Element 3
inline-block	Box 1 Box 2 Box 3
none	(Element is hidden)

THE BOX MODEL DIAGRAM



SHORTHAND SUMMARY

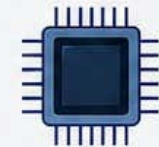
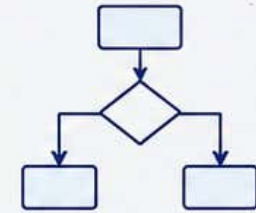
Property	1 Value	2 Values	3 Values	4 Values
margin	all sides	top & bottom left & right	top left & right bottom	top right bottom left
padding	all sides	top & bottom left & right	top left & right bottom	top right bottom left
border-width border-style border-color	all sides	top & bottom left & right	top left & right bottom	top right bottom left



QUICK NOTES

- Total Width = margin-left + border-left + padding-left + content width + padding-right + border-right + margin-right
- Total Height = margin-top + border-top + padding-top + content height + padding-bottom + border-bottom + margin-bottom
- Use border-radius to round corners.
- Use margin: auto to center block elements.
- Use display to control layout behavior of elements.

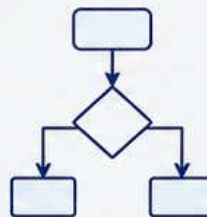
1010101010
0101010101
1010101010
0101010101



Unit - IV



10110
01001
10101



WHAT IS WEB HOSTING?

Web Hosting is a service that allows individuals and organizations to make their websites accessible on the Internet. A web hosting provider stores your website files (images, text, videos, code, etc.) on a powerful server connected to the Internet 24/7.



In simple words, web hosting provides the space on a server where your website lives.

NEED OF WEB HOSTING

- 1**  **Makes Your Website Accessible**
Hosting publishes your website on the Internet so people from anywhere in the world can visit it anytime.
- 2**  **Reliable and Available 24/7**
Good hosting ensures your website is available round the clock with minimal downtime.
- 3**  **Better Performance and Speed**
A good hosting provider offers fast servers and high uptime, which improves user experience and SEO ranking.
- 4**  **Security and Data Protection**
Hosting provides security features like SSL certificates, firewall, backups, and protection from attacks and data loss.
- 5**  **Supports Website Growth**
As your website grows, hosting allows you to upgrade resources (bandwidth, storage, etc.) to handle more traffic.
- 6**  **Professional Email & Services**
Hosting allows you to create professional email addresses (e.g., info@yourdomain.com) and use additional tools (databases, FTP, emails, etc.).
- 7**  **Cost-Effective Solution**
It is an affordable way for businesses, organizations or individuals to establish an online presence.

HOW IT WORKS (IN SHORT)



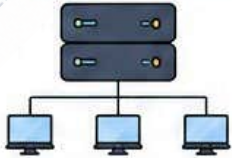
Example

When you visit www.example.com, your browser connects to the server where the website is hosted, gets the files, and shows you the website.

TYPES OF HOSTING PACKAGES

Hosting packages are different types of services that allow you to publish your website or application on the Internet.

1. SHARED HOSTING



Multiple websites share the same server resources like CPU, RAM, storage, bandwidth, etc.

Features

- Affordable
- Easy to use
- Suitable for small websites
- Limited resources
- Performance can be affected by other websites

Best For

Beginners, personal websites, blogs, small business websites.

Example Providers

Hostinger, Bluehost, SiteGround, GoDaddy

2. VPS HOSTING (VIRTUAL PRIVATE SERVER)



A virtual server created within a physical server. You get dedicated resources in a virtual environment.

Features

- More control and flexibility
- Dedicated (guaranteed) resources
- Better performance and security
- Scalable as your website grows
- More expensive than shared hosting

Best For

Growing websites, e-commerce sites, developers, medium businesses.

Example Providers

A2 Hosting, InMotion Hosting, Hostinger VPS, DigitalOcean

3. DEDICATED HOSTING



You get an entire physical server dedicated only to your website.

Features

- Full control over the server
- High performance and security
- Customizable hardware and software
- Handles high traffic
- Expensive

Best For

Large websites, high traffic sites, resource-intensive applications.

Example Providers

OVH, Hetzner, Liquid Web, HostGator Dedicated

4. CLOUD HOSTING



Your website is hosted on multiple connected servers (the cloud) to ensure reliability and scalability.

Features

- Highly reliable (no single point of failure)
- Scalable – pay for what you use
- High uptime
- Good performance
- Cost can vary based on usage

Best For

Growing websites, e-commerce sites, apps with variable traffic.

Example Providers

Amazon Web Services (AWS), Google Cloud, Microsoft Azure, Cloudways

5. WORDPRESS HOSTING



Specialized hosting optimized for WordPress websites.

Features

- One-click WordPress install
- Optimized speed and security
- Automatic updates and backups
- WordPress expert support
- Easy management

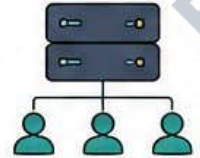
Best For

WordPress users, blogs, business websites on WordPress.

Example Providers

Bluehost WordPress, Kinsta, WP Engine, SiteGround WordPress

6. RESELLER HOSTING



Allows you to host multiple websites and resell space and bandwidth to others.

Features

- Create multiple hosting accounts
- White-label hosting
- Manage clients and websites
- Set your own prices
- Good profit potential

Best For

Web designers, developers, agencies, entrepreneurs.

Example Providers

HostGator Reseller, A2 Hosting Reseller, WHMCS, ResellerClub

QUICK COMPARISON

Hosting Type	Cost	Performance	Control	Scalability	Best For	Uptime	Technical Skills
Shared Hosting	Low	★☆☆☆☆	Low	Low	Beginners, Small Sites	99%	Basic
VPS Hosting	Medium	★★★★☆	Medium	Medium to High	Growing Sites	99.9%	Intermediate
Dedicated Hosting	High	★★★★★	High	High	Large Sites, High Traffic	99.9%	Advanced
Cloud Hosting	Medium to High	★★★★☆	Medium	Very High	Growing Sites, Apps	99.99%	Intermediate
WordPress Hosting	Medium	★★★★☆	Low to Medium	Medium to High	WordPress Users	99.9%	Basic
Reseller Hosting	Medium	★★★★☆	Medium	Medium	Resellers, Agencies	99.9%	Intermediate

NOTE

- ✓ Choose a hosting package based on your website needs, budget, and expected traffic.
- ✓ Start small and upgrade as your website grows.
- ✓ Always consider uptime, support, security and backup before selecting a hosting provider.



Remember: The right hosting package ensures your website is fast, secure, reliable and always available for your visitors.



REGISTERING DOMAINS

A **domain name** is your website's address on the Internet (e.g., www.example.com). Registering a domain means reserving that name so no one else can use it.

STEPS TO REGISTER A DOMAIN

- 1 Choose a Domain Name**
Select a name that is unique, easy to remember and related to your business or purpose.
- 2 Check Availability**
Use a domain registrar website to check if your desired domain name is available.
- 3 Select a Domain Extension**
Choose an appropriate TLD (Top Level Domain) such as .com, .org, .net, .in, etc.
- 4 Provide Registration Details**
Enter your details like name, email, phone number, address, etc. (These become WHOIS information).
- 5 Choose Registration Period**
Select how long you want to register the domain (1 year, 2 years, 5 years, etc.).
- 6 Make Payment**
Pay the domain registration fee. Prices vary depending on the domain extension and registrar.
- 7 Confirmation**
Once payment is successful, the domain is registered in your name and you will get a confirmation email.

THE DOMAIN REGISTRATION PROCESS



WHAT HAPPENS AFTER REGISTRATION?

- Your domain name is recorded in the WHOIS database.
- You become the legal owner for the chosen period.
- You can manage your domain (DNS settings, renewal, contact info, etc.) through your registrar account.

POPULAR DOMAIN EXTENSIONS

Extension	Purpose
.com	Commercial organizations, businesses (most popular)
.org	Organizations, non-profits
.net	Network services, technology
.gov	Government organizations
.edu	Educational institutions
.in	India
.co.in	Companies in India
.info	General information
.biz	Business
.io	Tech startups, developers

COMMON TERMS

- **Domain Registrar:** Company accredited to register domain names.
- **WHOIS:** Public database that stores domain owner information.
- **TLD:** Top Level Domain (e.g., .com, .org)
- **SLD:** Second Level Domain (e.g., example in example.com)
- **Subdomain:** Part before the domain (e.g., blog.example.com)

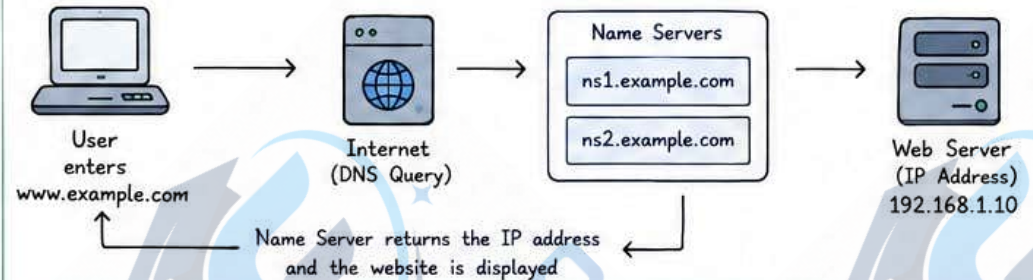
DEFINING NAME SERVERS

Name Servers (NS) are specialized servers that store DNS records for your domain and help in translating domain names into IP addresses.

WHAT IS A NAME SERVER?

- The Internet uses IP addresses (e.g., 192.168.1.10) to identify servers.
- Users remember names (e.g., www.example.com) instead of IP addresses.
- Name Servers act like a directory. They store DNS records and tell the Internet where your domain is hosted.

HOW NAME SERVERS WORK



TYPES OF NAME SERVERS

- **Primary Name Server:** The main server that contains the original DNS records.
- **Secondary (or Backup) Name Server:** A copy of the primary server. Used if the primary server is down.
- **Authoritative Name Server:** The server that has the final authority for the domain and provides the correct DNS information.

HOW TO DEFINE (UPDATE) NAME SERVERS FOR A DOMAIN

- 1 Login to your Domain Registrar account.
- 2 Go to Domain Management / My Domains.
- 3 Select the domain you want to manage.
- 4 Find the option "Name Servers" or "DNS Management".
- 5 Enter your Name Server details.
 - Enter your hosting provider's name servers (e.g., ns1.hosting.com, ns2.hosting.com) or
 - Enter custom name servers if you are using your own.
- 6 Save/Update the changes.
- 7 DNS changes may take some time to propagate (from a few minutes to 48 hours).

EXAMPLE

If your hosting provider gives:
ns1.hosting.com
ns2.hosting.com
You will update your domain's name servers as:

Name Server	Value
Name Server 1	ns1.hosting.com
Name Server 2	ns2.hosting.com

★ **Note:** Correctly defining name servers is important. If they are wrong, your website, emails and other services may not work.



Remember:

Register your domain name early and keep it renewed to protect your brand and avoid losing your online identity!

STEPS TO HOST THE WEBSITE

Hosting a website makes it available on the Internet for everyone to access.

1 PLAN YOUR WEBSITE



- Decide the purpose of your website.
- Plan the pages, content and features.
- Prepare your website files (HTML, images, videos, etc.).

2 REGISTER A DOMAIN NAME



- Choose a unique domain name for your website (e.g., www.yourname.com).
- Check availability.
- Register it through a domain registrar.
- The domain name will be your website address.

3 CHOOSE A WEB HOSTING SERVICE



- Select a hosting provider that suits your needs (shared, VPS, cloud, etc.).
- Compare plans, features, uptime, support and price.
- Purchase a hosting package.

4 CONFIGURE DOMAIN NAME SERVERS



- Your domain must point to your hosting server.
- Update the Name Servers in your domain registrar with the values provided by your hosting company.
- Changes may take some time to propagate.

5 PREPARE YOUR WEBSITE FILES



- Create your website using HTML, CSS, JavaScript, etc.
- Make sure all files (images, CSS, JS, etc.) are in proper folders.
- Test your website locally before uploading.

6 UPLOAD WEBSITE FILES TO THE SERVER



- Use File Manager in your hosting control panel (cPanel, etc.) or an FTP client (FileZilla).
- Upload all your website files to the public_html (or www) directory.

7 CREATE DATABASE (If Required)



- For dynamic websites (WordPress, etc.), create a database (MySQL).
- Note down the database name, username and password.

8 CONFIGURE YOUR WEBSITE



- Install necessary scripts (WordPress, PHP projects, etc.).
- Update database details in the configuration file.
- Set permissions if required.

9 TEST YOUR WEBSITE



- Visit your domain name in the browser.
- Check all pages, links, images and forms.
- Ensure everything works properly.

10 ENABLE SSL CERTIFICATE (SECURE YOUR SITE)



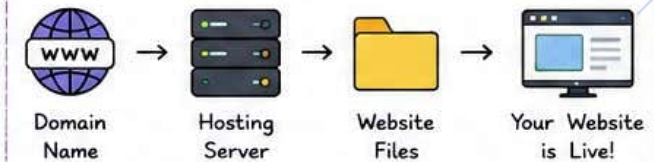
- Install an SSL certificate (usually free with your hosting).
- This enables <https://> and secures data between users and your website.

11 MAKE YOUR WEBSITE LIVE



- After testing and securing, your website is live on the Internet!
- Share your website URL with the world.

THE FLOW IN SHORT



IMPORTANT NOTES

- ✓ It may take a few minutes to a few hours for your domain changes to reflect worldwide.
- ✓ Keep regular backups of your website and database.
- ✓ Choose a reliable hosting provider for better performance, security and support.

EXAMPLE

You register domain: www.example.com
Your hosting provides Name Servers:
ns1.hosting.com, ns2.hosting.com
You set these in your domain registrar.

Now your website is accessible at <https://www.example.com>

